MOBILE APPLICATION AND DEVELOPMENT

Firebase Realtime Database & Authentication

Put your app’s data in the cloud, and let users sign in

Ekramul Haque Amin · Lecturer, AI & Data Science · Dhaka International University

TODAY'S ROADMAP

What We Will Cover

1. Why apps need a cloud, and what Firebase gives us
2. Connecting an Android project to Firebase
3. Realtime Database: the JSON tree, writing, reading and live updates
4. Showing cloud data in a RecyclerView
5. Firebase Authentication: register, login, logout and who owns what

LAST CLASS

Recap: Data That Lives on One Phone

SharedPreferences
Small key-value settings, such as a login flag

SQLite / Room
Tables, rows and queries stored on the device

The limit
Uninstall the app or lose the phone and the data is gone

Today’s question: how can two phones see the same data?

THE PROBLEM

Local Data Stays Local



Phone A saved two contacts



Phone B knows nothing about them

A server in the middle lets every phone read and write the same data.

THE FIX

Phones Share One Cloud Copy



Write on one phone, and the other updates within a moment.

PART 1 — FIREBASE

What Is Firebase?

Firebase is Google’s backend-as-a-service. Instead of writing and hosting your own server, you switch on ready-made services from a web console and call them from your app.

Authentication
Sign-up, login and user accounts

Realtime Database
A cloud JSON tree that syncs live

Cloud Firestore
Collections of documents, with queries

Cloud Storage
Photos, videos and other files

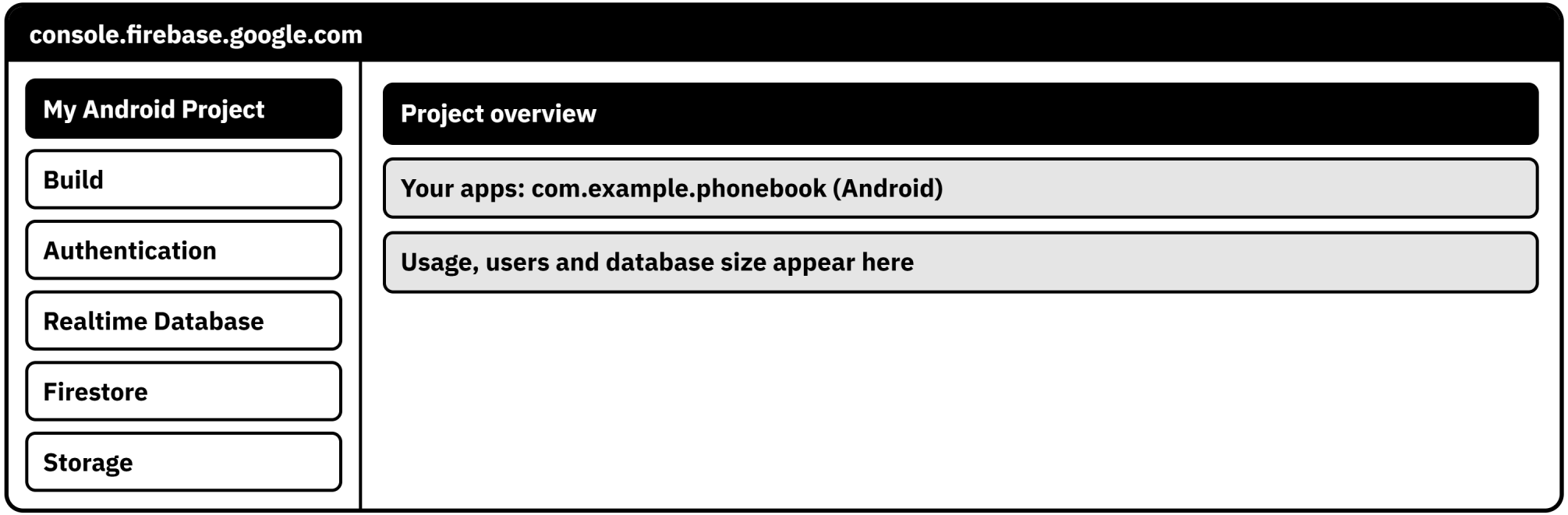
CHOOSING WHERE DATA LIVES

Local vs. Cloud Storage

	Local (SQLite/Room)	Cloud (Firebase)
Where	On the phone	On Google’s servers
Works offline	Always	Cached, then syncs
Shared between devices	No	Yes
Needs internet	No	Yes, to sync
Setup effort	Write the schema	Create a project

WHERE YOU MANAGE EVERYTHING

The Firebase Console



Sign in with a Google account at console.firebase.google.com. Everything is free to start.

SETUP, STEP 1

Create a Firebase Project

1. Open the Firebase console and press Add project
2. Give it a name, such as Phonebook
3. Google Analytics is optional. You can switch it off for class
4. Press Create project and wait a few seconds

SETUP, STEP 2

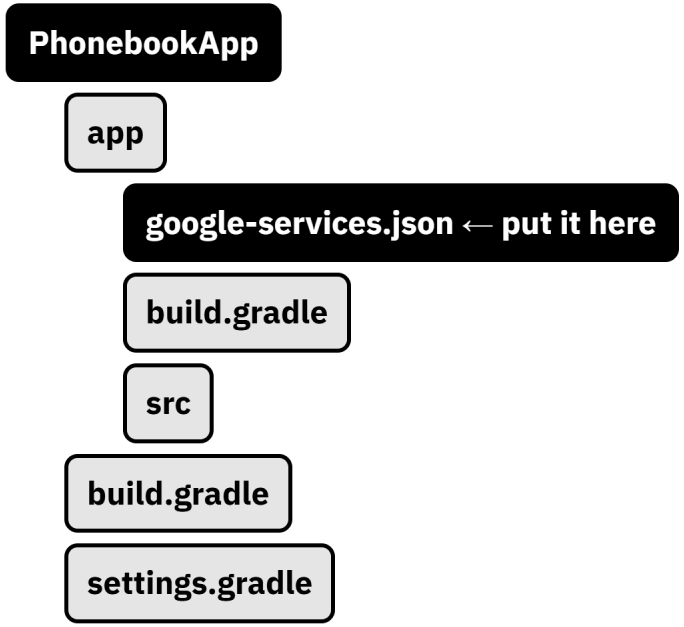
Register Your Android App

1. In the project overview press the Android icon
2. Package name: copy it exactly from your app, for example com.example.phonebook
3. Nickname is optional
4. SHA-1 is only needed for Google Sign-In, so skip it today
5. Press Register app

A wrong package name is the number one cause of “Firebase does not work” bugs.

SETUP, STEP 3

Add google-services.json



- Download the file from the console
- Switch Android Studio to the Project view, then drop the file into the app folder
- This file tells your app which Firebase project it belongs to
- Never post it on a public GitHub repository

SETUP, STEP 4

Add the Gradle Plugin and Libraries

```
// build.gradle (Project)
plugins {
    id 'com.google.gms.google-services' version '4.4.2' apply false
}

// build.gradle (Module: app)
plugins {
    id 'com.google.gms.google-services'
}
dependencies {
    implementation platform('com.google.firebase:firebase-bom:33.5.1')
    implementation 'com.google.firebase:firebase-auth'
    implementation 'com.google.firebase:firebase-database'
}
```

Version numbers change often. Copy the current ones from the Firebase docs. Press Sync Now when done.

SETUP, STEP 5

Internet Permission

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android">

    <uses-permission android:name="android.permission.INTERNET" />

    <application ... >
        ...
    </application>
</manifest>
```

Without this line the app cannot reach the cloud. Firebase usually adds it for you, but check.

A SHORTCUT

Tools → Firebase Assistant

Open it
Tools → Firebase shows a panel with every service

Follow the steps
Connect to Firebase, then Add the SDK to your app

It does the Gradle edits
Good for beginners. The manual way teaches what really changes

Either route ends in the same place: google-services.json plus Gradle dependencies.

BEFORE WRITING ANY CODE

Setup Checklist

1	Project created in the console
2	App registered with the exact package name
3	google-services.json is inside the app folder
4	Plugin and libraries added to both Gradle files, then Sync Now
5	INTERNET permission is in the manifest

PART 2 — REALTIME DATABASE

One Big JSON Tree

The Realtime Database stores everything as a single JSON tree. There are no tables or rows. You point to a path in the tree, then read or write the value there.

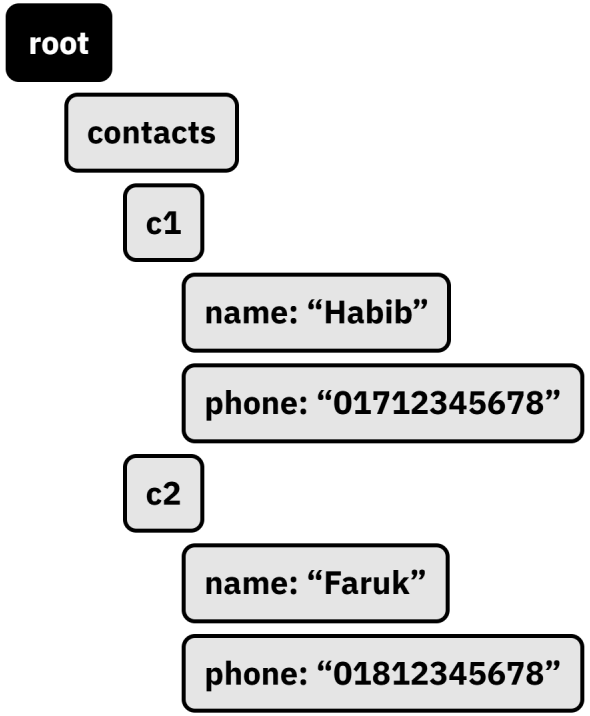
Path
contacts/c1/name

Value
“Habib”

Live
Listeners are told the moment a value changes

HOW THE DATA LOOKS

A Phonebook as a Tree



- Keys on the left, values on the right
- Every node has a path: contacts/c1/phone
- Nodes with children behave like folders
- Leaf nodes hold text, numbers or true/false

A DIFFERENT WAY TO THINK

Tree vs. Tables

Idea	SQLite / Room	Realtime Database
Structure	Tables and rows	Nested JSON tree
A record	A row with an id	A child node with a key
Find data	SQL query	Go to a path
Change	UPDATE or DELETE	setValue or removeValue

TURN IT ON

Create the Database

1. Console → Build → Realtime Database → Create Database
2. Pick the location nearest to your users
3. Start in test mode so the class can read and write freely
4. Copy the database URL if the console shows it

Test mode expires after 30 days and is open to everyone. Never ship it.

YOUR FIRST LINE OF CODE

DatabaseReference

```
// root of the database
DatabaseReference root = FirebaseDatabase.getInstance().getReference();

// the "contacts" node
DatabaseReference contacts =
        FirebaseDatabase.getInstance().getReference("contacts");

// go deeper
DatabaseReference habib = contacts.child("c1");
```

A reference is only a pointer to a path. Nothing is downloaded until you read or write.

DATA NEEDS A SHAPE

The Model Class

```
public class Contact {
    public String id;
    public String name;
    public String phone;

    public Contact() { } // required by Firebase

    public Contact(String id, String name, String phone) {
        this.id = id;
        this.name = name;
        this.phone = phone;
    }
}
```

Firebase needs a public empty constructor so it can rebuild the object when reading.

CREATE DATA

Writing with setValue()

```
DatabaseReference db =
        FirebaseDatabase.getInstance().getReference("contacts");

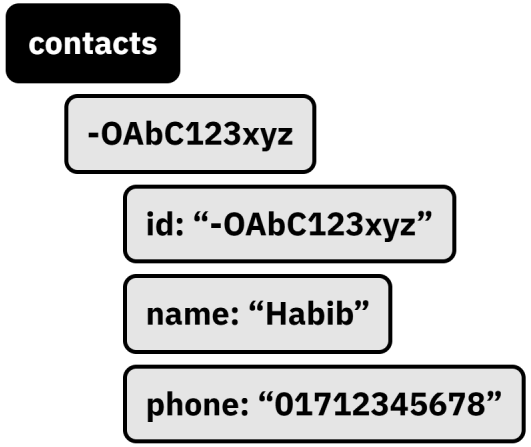
String id = db.push().getKey(); // unique key
Contact c = new Contact(id, "Habib", "01712345678");

db.child(id).setValue(c)
  .addOnSuccessListener(v → Toast.makeText(this, "Saved", Toast.LENGTH_SHORT).show())
  .addOnFailureListener(e → Log.e("DB", "Save failed", e));
```

push() makes a unique, time-ordered key, so two phones never overwrite each other.

WHAT JUST HAPPENED

The Tree After a Write



- A new child appears under contacts
- Its key was created by push()
- The Contact fields became child values
- The console shows the change live

CHANGE AND REMOVE

Update and Delete

```
// change one field
db.child(id).child("phone").setValue("01999999999");

// change several fields at once
Map<String, Object> changes = new HashMap<>();
changes.put("name", "Habib Rahman");
changes.put("phone", "01999999999");
db.child(id).updateChildren(changes);

// delete the whole contact
db.child(id).removeValue();
```

setValue on a node replaces everything under it. Use updateChildren to touch only some fields.

READING DATA

Read Once vs. Listen Forever

```
// read once
db.get().addOnCompleteListener(task → {
    DataSnapshot snap = task.getResult();
    // use snap here
});
```

```
// listen for every change
db.addValueEventListener(
    new ValueEventListener() {
        ...
    });
```

get() asks one time. addValueEventListener keeps the connection open and fires again on every change.

THE LIVE LISTENER

ValueEventListener

```
db.addValueEventListener(new ValueEventListener() {
    @Override
    public void onDataChange(@NonNull DataSnapshot snapshot) {
        list.clear();
        for (DataSnapshot child : snapshot.getChildren()) {
            Contact c = child.getValue(Contact.class);
            list.add(c);
        }
        adapter.notifyDataSetChanged();
    }
    @Override
    public void onCancelled(@NonNull DatabaseError error) {
        Log.w("DB", error.toException());
    }
});
```

UNDERSTANDING THE CALLBACK

What Is a DataSnapshot?

snapshot
A frozen copy of the data at that path, at that moment

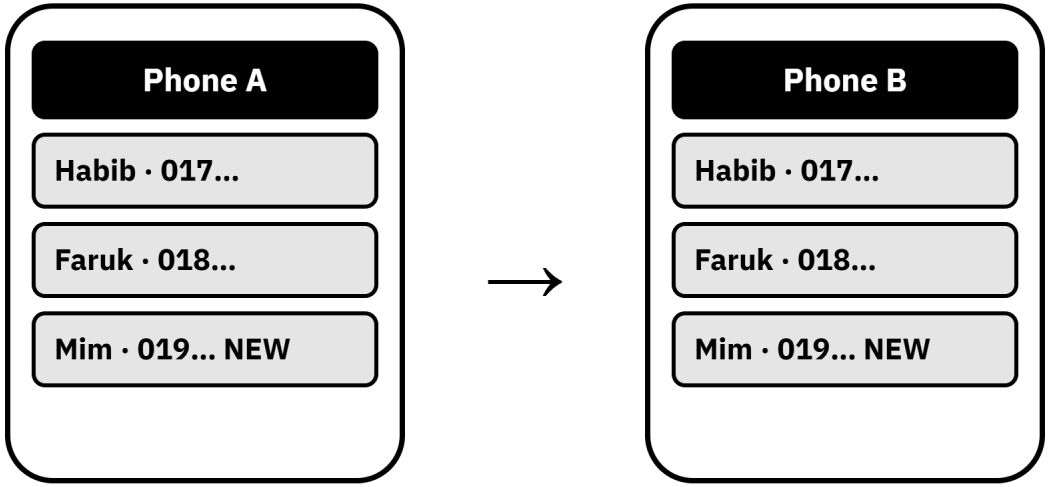
getChildren()
Loop over each child, such as every contact

getValue(Contact.class)
Turn a child into a Java object automatically

Clear your list first, or every change would add duplicates.

SEE IT LIVE

Two Phones, One Database



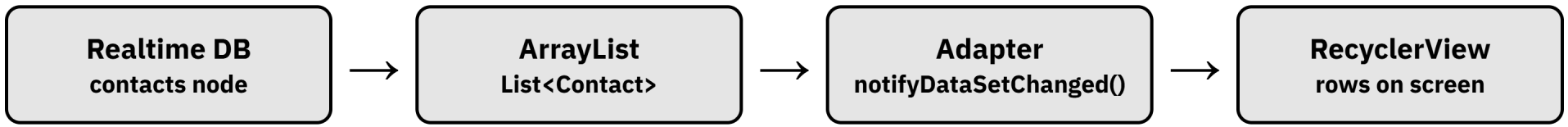
Phone A taps Add

Phone B updates by itself

No refresh button. The listener pushes the new value to every connected phone.

PUTTING IT ON SCREEN

Firebase → List → RecyclerView



You already know the adapter from Week 7. Only the data source changed: from a hard-coded list to Firebase.

WIRING IT TOGETHER

In onCreate()

```
RecyclerView rv = findViewById(R.id.recyclerView);
rv.setLayoutManager(new LinearLayoutManager(this));

list = new ArrayList<>();
adapter = new ContactAdapter(this, list);
rv.setAdapter(adapter);

db = FirebaseDatabase.getInstance().getReference("contacts");
db.addValueEventListener(listener); // from the previous slide
```

GOOD MANNERS

Stop Listening When You Leave

```
private ValueEventListener listener;

@Override
protected void onStop() {
    super.onStop();
    db.removeEventListener(listener);
}
```

A forgotten listener keeps downloading data and can leak memory. Register in onStart, remove in onStop.

A NICE BONUS

Offline Persistence

```
// call once, before any other Firebase use
FirebaseDatabase.getInstance().setPersistenceEnabled(true);
```

What it does
Keeps a local copy of recently used data

When offline
Writes are queued and sent when the phone reconnects

Call it once, for example in a custom Application class, before using any database reference.

WHO MAY READ AND WRITE?

Security Rules

```
{
  "rules": {
    ".read": "auth ≠ null",
    ".write": "auth ≠ null"
  }
}
```

Test mode
Anyone on the internet can read and write

auth != null
Only signed-in users get through

Rules live in the console. They run on Google’s servers, so users cannot bypass them.

PART 3 — AUTHENTICATION

Who Is Using the App?

Without accounts, everyone sees and edits everything. Authentication proves who a user is, so each person gets their own data and your database rules have someone to check.

Register
Create an account with email and password

Login
Prove who you are

Session
Firebase remembers the user between app launches

THE BIG PICTURE

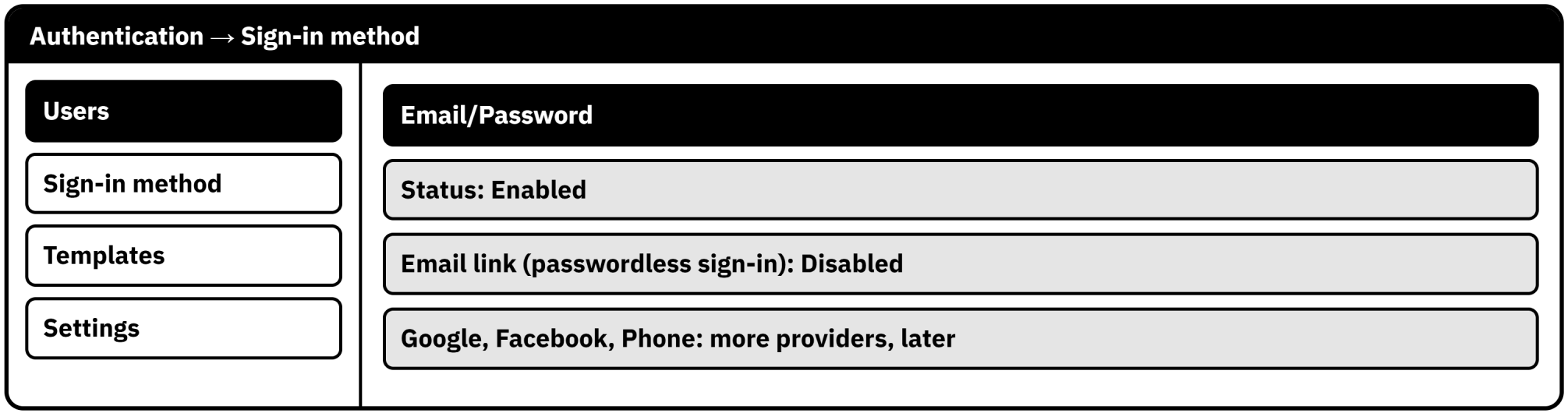
How Email Login Works



You never store or see the password yourself. Firebase handles hashing and storage.

SWITCH IT ON

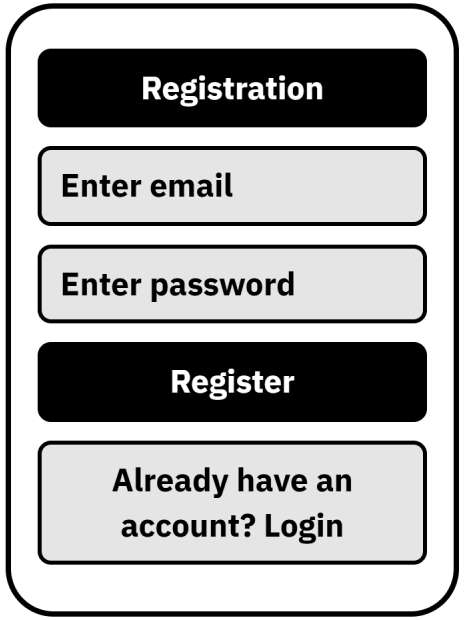
Enable Email/Password Sign-in



Console → Build → Authentication → Get started → Sign-in method → Email/Password → Enable → Save.

THE SCREEN

Registration Layout



- Two fields: email and password
- TextInputEditText with inputType textEmailAddress and textPassword
- A Register button and a link to the Login screen
- The same layout idea as our login screen in Week 4

CREATE THE ACCOUNT

createUserWithEmailAndPassword()

```
mAuth = FirebaseAuth.getInstance();

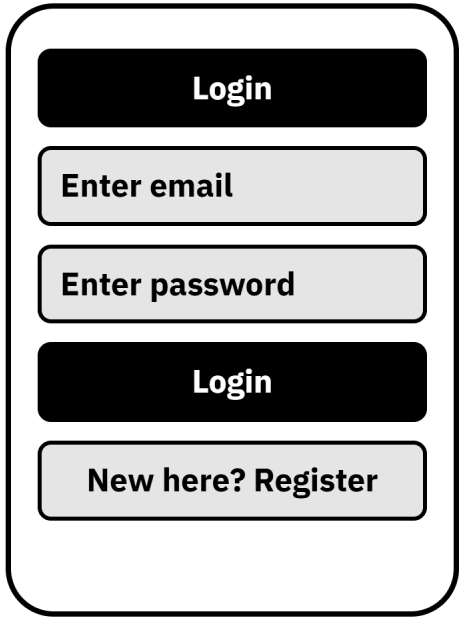
btnRegister.setOnClickListener(v → {
    String email = etEmail.getText().toString().trim();
    String pass = etPassword.getText().toString().trim();

    if (TextUtils.isEmpty(email)) { etEmail.setError("Email is required"); return; }
    if (TextUtils.isEmpty(pass)) { etPassword.setError("Password is required"); return; }

    mAuth.createUserWithEmailAndPassword(email, pass)
         .addOnCompleteListener(task → {
             if (task.isSuccessful()) {
                 Toast.makeText(this, "Account created", Toast.LENGTH_SHORT).show();
             } else {
                 Toast.makeText(this, "Failed: " + task.getException().getMessage(),
                         Toast.LENGTH_LONG).show();
             }
         });
});
```

SIGN IN

signInWithEmailAndPassword()




```
mAuth.signInWithEmailAndPassword(email, pass)
     .addOnCompleteListener(task → {
         if (task.isSuccessful()) {
             startActivity(new Intent(this, MainActivity.class));
             finish();
         } else {
             Toast.makeText(this, "Wrong email or password",
                     Toast.LENGTH_SHORT).show();
         }
     });
```

finish() removes the login screen from the back stack.

REMEMBER THE USER

Already Signed In?

```
@Override
public void onStart() {
    super.onStart();
    FirebaseUser user = mAuth.getCurrentUser();
    if (user ≠ null) {
        startActivity(new Intent(this, MainActivity.class));
        finish();
    }
}
```

Firebase keeps the session, even after the app closes. Same idea as the SharedPreferences login flag, but handled for you.

WHO IS THIS?

Reading the FirebaseUser

```
FirebaseUser user = FirebaseAuth.getInstance().getCurrentUser();

String uid = user.getUid(); // unique id, never changes
String email = user.getEmail();
```

The uid is the key to everything: use it to give every user a private part of the database.

LEAVING

Sign Out

```
btnLogout.setOnClickListener(v → {
    FirebaseAuth.getInstance().signOut();
    startActivity(new Intent(this, LoginActivity.class));
    finish();
});
```

After signOut(), getCurrentUser() returns null.

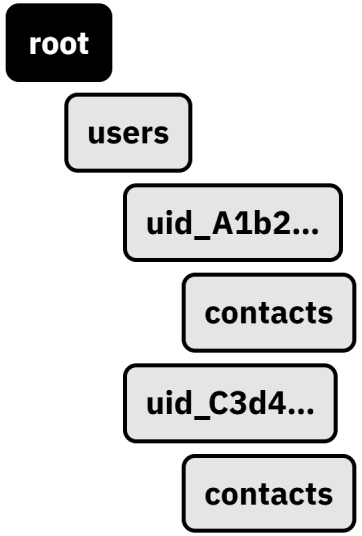
WHEN IT FAILS

Common Authentication Errors

Message	Meaning	Fix
Password should be at least 6 characters	Weak password	Validate length first
The email address is badly formatted	Not a valid email	Check the text before sending
The email address is already in use	Account exists	Send the user to Login
The supplied auth credential is incorrect	Wrong email or password	Show a clear message
A network error occurred	No internet	Check connection, add retry

EACH USER OWNS A BRANCH

Per-User Data with the uid




```
String uid = FirebaseAuth.getInstance()
        .getCurrentUser().getUid();

DatabaseReference myContacts =
    FirebaseDatabase.getInstance()
        .getReference("users")
        .child(uid)
        .child("contacts");
```

LOCK IT DOWN

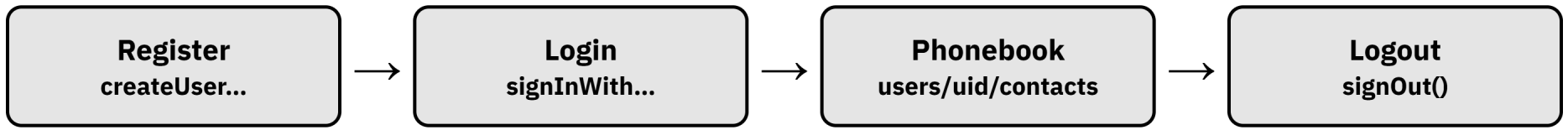
Rules That Use the uid

```
{
  "rules": {
    "users": {
      "$uid": {
        ".read": "$uid === auth.uid",
        ".write": "$uid === auth.uid"
      }
    }
  }
}
```

Each signed-in user can reach only users/<their own uid>. Everything else is blocked by default.

PUTTING TODAY TOGETHER

The Complete App Flow



This is exactly the Phonebook app you will build in the lab.

WHEN NOTHING WORKS

Troubleshooting Firebase

App crashes at start
google-services.json missing or in the wrong folder

Nothing is saved
Check rules, internet permission and the database URL

Empty list
Model class needs a public empty constructor and matching field names

Auth always fails
Email/Password provider is not enabled in the console

LEARN FROM THESE NOW

Common Mistakes

Not clearing the list
Every change adds duplicates

Leaving test mode on
Anyone can erase your database

Hard-coded keys
Use push() for unique ids

Updating the UI off the callback
Data arrives later. Update inside onDataChange

BEFORE YOU LEAVE TODAY

Recap Checklist

1. Can you list the five setup steps to connect Firebase?
2. How does the Realtime Database store data, and how is that different from a table?
3. What is the difference between get() and addValueEventListener()?
4. Why does a model class need an empty constructor?
5. How do the uid and security rules keep each user’s data private?

WHERE WE GO FROM HERE

Next Steps

Next theory: Firestore
Collections, documents, queries and a richer data model

This week: Lab
Phonebook app with Firebase and Firestore: register, login and a live contact list

Before the lab: create your Firebase project and download google-services.json.

Questions?

Firebase setup | Realtime Database | Listeners | Authentication

Ekramul Haque Amin

Lecturer, AI & Data Science · Mobile Application and Development (0613-409)